

Boundary Case: Schema Evolution / Substrate Migration as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the schema-evolution / substrate-migration boundary case as a standalone architectural treatment, articulating how a CKS-coherent substrate handles evolution of its data-structure schemas across deployment lifetime while preserving path retraceability, reproducibility, and the determinism contract.

Abstract

A long-running CKS deployment encounters a structural pressure no single foundational commitment in the source paper resolves on its own: substrate data-structure schemas evolve. New fields are added; old fields are deprecated; data types change; relationship structures shift. Historical state — written under prior schemas — remains substrate-resident under the path-retraceability commitment. New operations expect the current schema. The deployment must operate coherently across schema versions while preserving historical context. This note formalizes the boundary as a standalone architectural treatment: schema changes are governance events authored under the orchestration-rule-authoring commitment; the schema active at write time is recorded in substrate provenance; historical state retains its original schema by default; current operations interact with historical-schema state through one of four human-chosen compatibility patterns (in-place backward compatibility, versioned readers, explicit re-processing, or federation); schemas referenced by historical provenance remain authoritative content even after deprecation for new use; replay against historical executions uses the schema active at execution time. The pattern parallels rule-version coexistence for cell behavior — both apply non-retroactivity, record version in provenance, and require human authoring for migration. Nine anti-patterns would each violate the architecture in a specific way. As the closing note of Phase A6 and of Series A, this note also names the cumulative coverage Series A's 197 notes provide of Paper 1 derivations.

1. Why the schema-evolution boundary needs to be formalized

Schema evolution is operationally inevitable for any CKS deployment with a non-trivial lifetime. Authored content categories expand; field semantics get refined; new relationship types become useful; older field shapes become inadequate. None of these are unusual events, and each stresses the substrate's architectural

commitments simultaneously, in a way the foundational commitments treat individually but not as a coherent boundary.

The boundary is non-obvious in a specific way. None of the source paper's commitments is *about* schema change; each is about a property the substrate must hold at a given time. Path retraceability (§5) is a property of substrate state at any moment. The determinism contract is a property of reads and writes against the substrate as it stands. The substrate-as-source-of-truth commitment (§11.3) is a property of where the answer to *what is currently the case* lives. Each is well-defined under a fixed schema. What none addresses directly is what becomes of those commitments when the schema itself changes mid-lifetime, while content written under the prior schema continues to exist.

Naming the boundary as standalone matters because the wrong treatment unwinds exactly the commitments the source paper defends. As the fifteenth and final Phase A6 note, this also closes the boundary-case enumeration of Series A.

2. The boundary case scenario

A CKS deployment runs for an extended period and accumulates substrate content under one or more schemas. At some point, a schema change is authored: a new field is added, an existing field's allowable values change, a relationship type is renamed, or an index is restructured. The change applies to substrate writes from a defined moment onward. Historical content — already written under the prior schema — remains in the substrate, addressable, with provenance attached. Path retraceability requires that historical state remain reachable; the determinism contract requires that historical reads yield the historical answer; reproducibility against historical executions requires that replay yield equivalent substrate-write behavior.

The pressure that makes this non-obvious is simultaneity. New operations want the new schema; historical operations are recorded against the old; the substrate carries both at once. Migrating everything to the new schema breaks retraceability against old executions; refusing to migrate prevents current operations from using the new schema's affordances against historical content; allowing LLM-mediated translation collapses the determinism contract into model-output non-determinism. The architecture must support a coherent answer that does not pick any of those failures.

3. Which architectural commitments the boundary stresses

Five commitments are stressed simultaneously when schemas evolve.

Path retraceability. Historical content was written under prior schemas; its provenance references a schema active at write time. If schemas are silently migrated or schema definitions are deleted, the antecedent path either no longer matches what was written or runs through definitions the substrate does not carry.

Reproducibility. Replay against historical executions requires that the substrate state read by those executions, and the schema under which it was read, are reconstructable. If a current-schema reader silently translates historical state, replay is no longer reproducible at the substrate-write layer.

The determinism contract. Read determinism, write determinism modulo LLM, and write addressability all assume a stable representation. Schema change does not abolish the contract; it requires that the contract's guarantees be specified per schema version. Reads of the same historical state under the historical schema must yield the same content; reads under a different schema may yield different content, and that difference must be observable rather than silent.

Human-governed authority. Schema changes are not technical conveniences the substrate may apply to itself; they are architectural decisions about what the substrate carries and how. Like orchestration rules, schemas must be authored by humans. The labor of drafting may be allocated to LLM-assisted tools; the authority over the schema as it takes effect is not.

Rule retroactivity. Schema definitions are part of the substrate's authoritative specification of what rules apply. Like other rule changes, they are non-retroactive by default: a schema change applies to writes from its effective time onward, not to writes that already exist under the prior schema. Extending the rule-retroactivity treatment to schema changes is the structural move that makes coexistence of historical and current schemas an architectural property rather than an ad-hoc concession.

4. The architectural treatment

The treatment is a composition of established commitments applied to schema as a category of authoritative content.

Schema changes are authored as governance events. A schema change is drafted, reviewed, and put into effect under the orchestration-rule-authoring commitment. The schema definition is substrate-resident content with a writer, a timestamp, and a rationale; the change has a moment of effect after which new writes use the new schema. No automated process — vendor tool, LLM agent, compliance framework, or migration utility — can put a schema change into effect outside this authority.

Provenance carries schema version. The provenance metadata fields the substrate requires include the schema version active at write time. When historical content is read, the schema version under which it was written is recoverable from its own provenance, so a reader can determine which schema applies to any piece of content by reading the substrate alone.

Historical state retains its original schema. The default treatment of pre-existing content under a schema change is non-retroactive: content remains as written, with its original schema version recorded. No silent migration occurs. This preserves path retraceability because the antecedent path runs through content as it was actually written.

Schemas referenced by provenance remain authoritative content. Schema definitions no longer used for new writes are not deleted; they remain substrate-resident as long as any historical content references them. Deprecation of a schema for new use is one event; erasure of the schema definition is a different event, and the second is not architecturally permitted while the first holds. Schema versions are themselves part of the substrate's source-of-truth status.

Compatibility is human-authored as one of four patterns. When current operations need to interact with historical-schema content, the deployment authors a rule under the orchestration-rule-authoring commitment specifying how. Four patterns are admissible; the choice is a deployment decision:

- *In-place backward compatibility.* Current rules read multiple schema versions natively, dispatching by the schema version recorded in provenance.
- *Versioned readers.* Each schema version has a versioned interpretation rule that translates historical content to a normalized form at read time. The translation rule is itself human-authored substrate content, and its outputs are reproducible because the rule is fixed.

- *Explicit re-processing.* Where the deployment chooses to materialize historical content under the current schema, re-processing is authored as a substrate write that creates *new* state alongside the historical state, with provenance referencing both the original content and the rule under which the re-processing occurred. The original is not overwritten.
- *Federation.* Different substrate partitions operate under different schemas, with rules specifying how cross-partition reads and writes occur. Appropriate when a deployment-wide schema change is impractical and partial migration is preferred.

Each pattern preserves retraceability and the determinism contract because each is itself authored substrate content with addressable provenance.

Replay uses the historical schema. Reproducibility against historical executions reads historical state under the schema active at execution time. The schema version is recoverable from the provenance of the state being read; replay is bound to that version, not to the current one. Replay reproduces what was, not what is.

The pattern parallels the rule-version-coexistence treatment: rule-version compatibility addresses cell behavior across rule versions; schema-version compatibility addresses substrate data structure across schema versions. Both apply non-retroactivity, both record version in provenance, both require human authoring for migration. The two are distinct architectural concerns operating on parallel principles.

5. Anti-pattern treatments that violate the architecture

Nine anti-patterns recur at this boundary.

Auto-schema-migration without human authoring. A vendor tool, automated pipeline, or LLM agent applies a schema migration without an authored rule. The change has no writer attribution recoverable from substrate content; the act of migration cannot be retraced as a governance event. Violates the human-governed and rule-authoring commitments.

Schema erasure. A schema definition referenced by historical provenance is deleted; historical content becomes uninterpretable in its own terms. Violates path retraceability — the antecedent path runs through schema content the substrate does not carry.

Silent schema breakage. Current operations read historical content under the current schema without checking the schema version recorded in provenance. The read either silently produces wrong values (a deprecated field interpreted under new semantics) or fails opaquely. Violates read determinism: the read result is not a function of substrate state alone.

Vendor-managed schema evolution. A vendor system controls schema migration outside the substrate's governance — a managed catalog, a vendor-hosted schema registry, or a runtime-middleware schema service mutates the substrate's representation in response to events the substrate did not author. Violates tool-agnosticism and the human-governed commitment.

LLM-mediated schema translation. An LLM is used to translate historical state into current-schema form at read time without an authored translation rule. The translation is non-deterministic; reproducibility collapses; the historical state's meaning becomes a function of the LLM's output rather than of the substrate's content. This is a category-specific instantiation of the broader anti-pattern previously formalized as silent LLM-mediated transformation of authoritative content.

Compliance-framework-imposed schema migration. An external compliance framework mandates a schema change and applies it directly to the substrate, bypassing human authoring inside the deployment's governance. The change may be substantively necessary; the architectural failure is that it is not authored as a substrate event under the deployment's authority.

Current-schema-applied-to-historical-state-without-version-check. Operations apply the current schema to historical content without consulting the schema version recorded in provenance. Even when no harm results in a specific case, the architecture loses the guarantee that schema-version-aware behavior is the default rather than a discretionary check.

Vendor-specific schema features. A schema is designed around features available only in a specific vendor's data structure (proprietary indexing, vendor-specific relationship types, vendor-locked validation primitives). The schema cannot be re-instantiated in any other commodity environment. Violates tool-agnosticism: the substrate's representational form is no longer realizable under the three minimal requirements.

Schema-version erasure from provenance. Provenance metadata is reduced to a form that omits the schema version. Historical content remains readable, but the schema under which it was written is no longer recoverable from substrate content alone, and schema-version-aware behavior becomes impossible. Violates path retraceability on the schema axis.

Each is a distinct way the boundary fails when the architectural treatment is not applied. Naming them is what allows downstream remediation to be precise rather than diffuse.

6. Operational implications

Schema design is forward-looking by necessity. Initial schemas should anticipate that they will not be the last; field-level versioning, optional rather than mandatory new fields where possible, and provenance fields that include schema version are baseline design choices rather than late-stage additions.

Multiple compatibility patterns are admissible in the same deployment — in-place backward compatibility for some content categories, versioned readers for others, federation for still others. The choice is a deployment decision authored as substrate content; the architecture does not impose a single pattern.

Schema versions accumulate, and schemas referenced by historical provenance remain substrate-resident. Where the deployment is capacity-constrained, archival treatments — separating historical schema content into archive partitions while preserving its addressability — apply, paralleling the substrate-near-capacity treatment for ordinary content. Archival is not erasure.

Replay tooling, where present, must be schema-version-aware: a replay that does not consult the schema version recorded in the substrate it replays against cannot reproduce the historical execution faithfully. The boundary's compliance is testable through reproducibility-across-schema-versions tests against historical executions whose schemas differ from the current, and through provenance-completeness tests verifying that schema version is among the recorded provenance fields.

7. Limits of the architectural treatment

The treatment specified here applies to schema evolution as a category of architectural change. It does not address rule-version compatibility for cell behavior, which is the parallel concern under a separate boundary

treatment. It does not address ordinary rule revisions whose effect does not change schema structure — those follow the rule-retroactivity treatment without invoking schema-coexistence machinery. It does not address substrate near-capacity per se, though it intersects with capacity when migration creates additional state alongside historical state. It does not specify migration tooling, schema-versioning frameworks, or data-evolution products; the treatment is what the substrate must do, not what tools must be procured to make it do so.

The treatment also does not eliminate the labor of schema design. Authoring a schema change well — anticipating compatibility implications, choosing the right compatibility pattern for each content category, ensuring the new schema preserves what historical readers need — remains substantial work. The architecture provides the contract under which that work yields a coherent system; it does not make the work cheap.

8. Operational test

A system handles the schema-evolution boundary in the CKS sense if and only if all of the following are true at all times during the substrate's existence:

1. Schema definitions are substrate-resident content with their own provenance, including writer attribution and effective-time information.
2. Provenance metadata for every piece of substrate content includes the schema version active at write time.
3. Schema changes take effect through human-authored rules under the orchestration-rule-authoring commitment; no automated, vendor-managed, or LLM-mediated process puts a schema change into effect outside that authority.
4. Historical substrate content is non-retroactively preserved under the schema active at write time; no silent migration of historical state occurs.
5. Schema definitions referenced by historical provenance remain substrate-resident, even when no longer used for new writes.
6. Compatibility between current operations and historical-schema content is handled through one of the human-authored patterns named in §4; LLM-mediated schema translation outside an authored rule is excluded.
7. Replay against historical executions uses the schema version recorded in the substrate state being replayed against, not the current schema.

A system that fails any of (1)–(7) handles schema evolution in some other sense and may have other valuable properties, but does not satisfy the architectural treatment specified here.

9. Why naming this boundary as standalone matters, and what closes with it

Schema evolution is the substrate-level analogue of rule-version coexistence. The two operate on parallel principles — non-retroactivity, version-recorded provenance, human-authored migration where chosen — at different layers of the architecture. Naming the schema-version concern as a standalone boundary case prevents three failure modes that would otherwise be invisible: the temptation to handle schema migration as a vendor or framework feature outside substrate governance; the temptation to treat schema evolution as an exception to retraceability rather than as a structurally retraceable event; and the temptation to admit silent

LLM-mediated translation across schemas as a convenience. Each is a real failure mode in current practice; each is identifiable only when the boundary is named.

This note closes Phase A6, the boundary-case enumeration of Series A. With Phase A6 complete, Series A's coverage of Paper 1 derivations comprises 197 notes spanning six phases: sixteen foundational architectural commitments (Phase A1); ninety-five operational variants and decompositions (Phase A2); twenty-five anti-pattern formalizations (Phase A3); thirty composition pairs producing emergent properties (Phase A4); sixteen operational tests as standalone (Phase A5); and fifteen boundary cases (Phase A6). Each note states one derivation precisely; the chain across all six phases is what makes the prior-art coverage comprehensive rather than concentrated.

Subsequent project work proceeds along parallel structures: Series B for Paper 2's instinct-and-reasoning-separation derivations (~210 notes following the same six-phase structure); Series C for the Paper-2-to-Paper-1 cross-derivation covering inheritance edges (~30 notes); Series D, E, and F for Paper 3's inter-Self coordination derivations (~220–260 notes). The architectural pattern — comprehensive enumeration of derivations under each paper's foundational commitments — carries forward; A6.15's place in the chain is the closing note for Paper 1 and the structural template the subsequent series instantiate.

Subsequent work that adopts CKS, extends it to long-running deployments with evolving substrate schemas, or argues against the architecture should treat the schema-evolution boundary in the sense formalized here. Subsequent work that handles substrate-data-structure evolution differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: Schema Evolution / Substrate Migration as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.